

Linear Programming in Quarto

Your Name

Linear Programming Model

$$\begin{aligned} \max \quad & \sum_{j=1}^n c_j x_j \\ \sum_{j=1}^n a_{ij} x_j & \leq b_i, \quad i = 1, \dots, m \\ x_j & \geq 0, \quad j = 1, \dots, n \end{aligned} \tag{1}$$

Equation 1 is a linear programming model with n decision variables x_j and m constraints. The coefficients c_j represent the contribution of each variable to the objective, while a_{ij} and b_i define the constraints.

We simulate data.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
n = 20
m = 10

A = rng.uniform(0, 10, (m, n))

print(A)

plt.figure(figsize=(5, 6))
plt.imshow(A, aspect="auto", interpolation="nearest", cmap="viridis")
plt.colorbar(label="Value")
plt.xlabel("Column")
plt.ylabel("Constraint Index")
#plt.xticks([0, 1], ["Intercept", "x"])
plt.title("Constraint matrix A")
plt.show()
```

[7.73956049 4.3887844 8.5859792 6.97368029 0.94177348 9.75622352
 7.61139702 7.86064305 1.28113633 4.50385938 3.70798024 9.26764989
 6.4386512 8.22761613 4.43414199 2.27238722 5.54584787 0.63817256
 8.27631172 6.31664399]
 [7.5808774 3.54525968 9.70698024 8.93121121 7.78383497 1.94638708
 4.66721004 0.43803766 1.54289492 6.83048953 7.44762156 9.67509732
 3.25825358 3.70459706 4.69555811 1.89471359 1.29921505 4.75704926
 2.26909349 6.69813995]
 [4.37151919 8.32678196 7.00265102 3.12366641 8.32259801 8.04764357
 3.87478379 2.88328104 6.82495504 1.39752484 1.99908202 0.0736227
 7.86924378 6.64850857 7.05165379 7.80729031 4.58915776 5.68741196
 1.39796998 1.14530074]
 [6.68402962 4.71096206 5.65236106 7.64998857 6.3471832 5.53579401
 5.59207161 3.03950098 0.30817835 4.36717389 2.14584673 4.08528644
 8.53403073 2.33939486 0.58302742 2.81383892 2.93593758 6.61916515
 5.57032152 7.83898209]
 [6.6431354 4.06386861 8.14020385 1.6697292 0.22712073 0.90047861
 7.22359351 4.6187723 1.61271779 5.01044775 1.52312103 6.96320375
 4.46156276 3.81021226 3.01512089 6.30282593 3.61812611 0.87649919
 1.18005902 9.61897665]
 [9.08580691 6.99707134 2.65869961 9.69176377 7.78750904 7.16890189
 4.49361502 2.72241562 0.96390962 9.02602397 4.5577629 2.02363365
 3.05956624 5.79219569 1.76772783 8.56614284 7.5851953 7.19462956
 4.3209304 6.27308841]
 [5.84097969 6.49846602 0.84444321 4.15807402 0.41614174 4.93990819
 3.29861212 1.44524189 1.03402968 5.87644572 1.70592969 9.25120118
 5.8106114 3.46869805 5.90915491 0.22803871 9.58559213 4.82303437
 7.82735227 0.8273]
 [4.86658331 4.90706994 9.37826455 5.71728052 4.73489401 2.66975663
 3.31568997 5.20672402 4.3891146 0.2161208 8.26291924 8.96160772
 1.40249089 5.54036144 1.08575741 6.72240093 2.81233784 6.59422635
 7.26994614 7.68647492]
 [1.07740946 9.16011845 2.30213991 0.37412556 5.54852469 3.70922284
 8.29789743 8.08251472 3.17138893 9.52899395 2.90917838 5.15057129
 2.55965091 9.3604357 1.64607818 0.44910619 4.3509706 9.92375564
 8.91677266 7.48608019]
 [8.90792491 8.9344664 5.1885836 3.15929052 7.72012432 6.61661263
 3.73657729 0.94466668 7.46789611 2.62460516 9.36813151 2.40970575
 1.22757932 8.31112672 1.53284317 1.79268308 5.99382792 8.74562041
 1.96434666 3.10323673]]

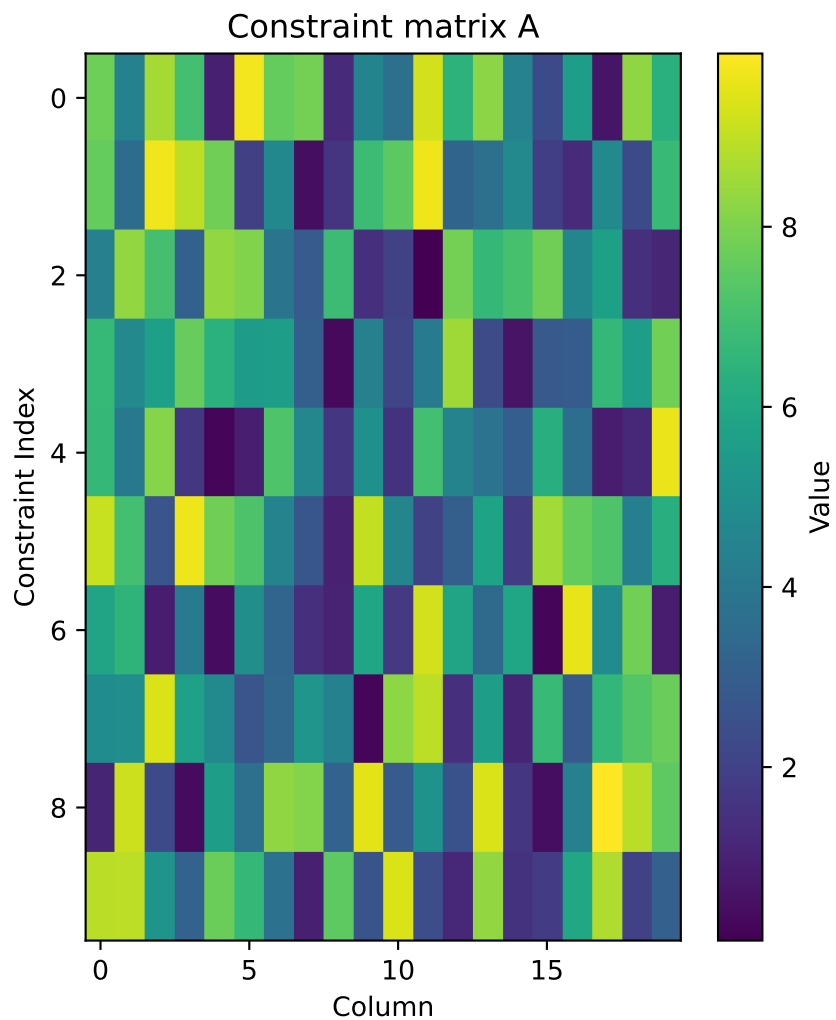


Figure 1: Simulated data for the constraint matrix.

Results

As shown in Figure 1, the estimates are $n = 20.00$ and $m = 10.00$.